Computational Physics Exercise 6

Student ID: 2024141230044

Name: 范奕

Problem 1

The original classroom problem was to use a high-order interpolation polynomial to observe the hazard of extrapolation. In class, the teacher first showed an example based on the first eight points of the USA population data. A seventh-order polynomial was constructed from 1920 to 1990 and then used to extrapolate later years. The result already suggested that although interpolation may pass through all known points exactly, extrapolation outside the data interval can become very unreliable.

After that, the teacher asked us to extend this idea and do a more detailed experiment on the birth-population data of a certain country from 2010 to 2025. The goal is to predict the value in 2026 using different interpolation windows. More specifically, for each interval

$$2010\text{–}2025,\ 2011\text{–}2025,\ 2012\text{–}2025,\ \ldots,\ 2024\text{–}2025,$$

I use all available points in that interval to construct an N-point Newton interpolation polynomial, and then evaluate it at 2026. In addition, in order to have a smoother reference, I also use a quadratic least-squares fitting curve to predict the 2026 value.

The dataset used in this report is

Year	2010	2011	2012	2013	2014	2015	2016	2017
Value	1588	1600	1635	1640	1687	1655	1786	1723

Year	2018	2019	2020	2021	2022	2023	2024	2025
Value	1523	1465	1200	1062	956	902	954	792

The purpose of this experiment is not simply to find one "best" answer, but to compare different prediction schemes and observe how sensitive high-order interpolation becomes when the data have obvious fluctuations and turning points.

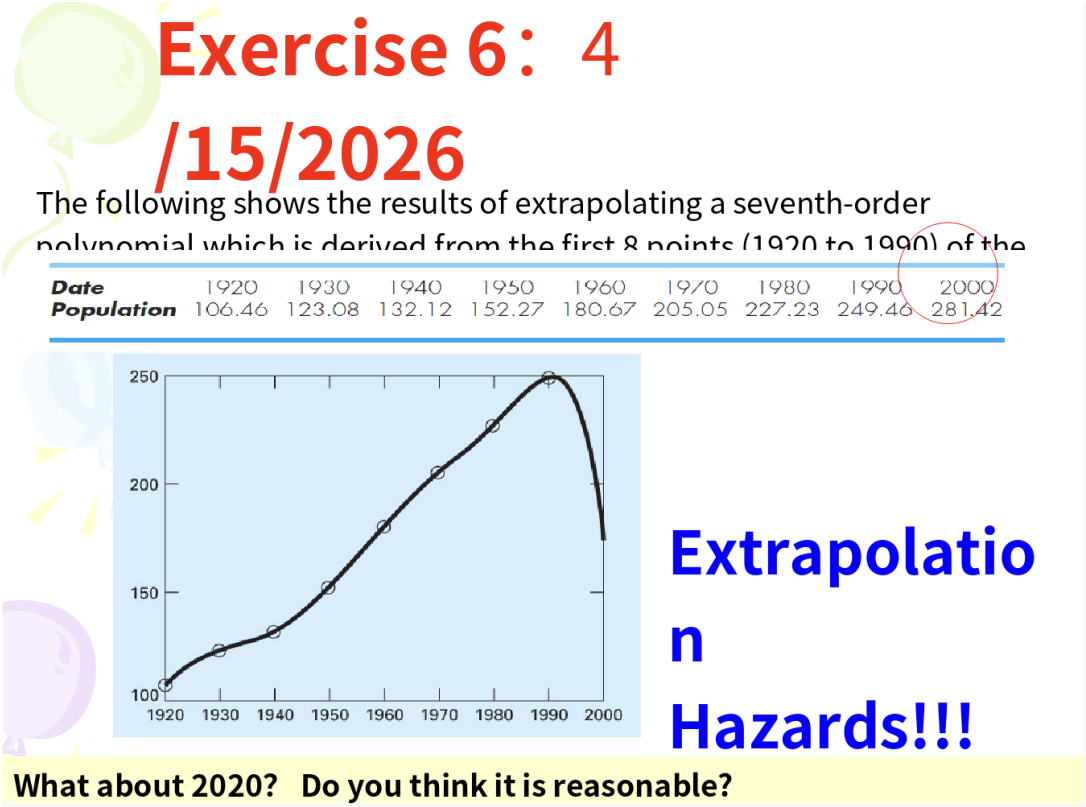

图 1: Original classroom slide illustrating extrapolation hazards.

目录

1 Part 1. Source Code 4

2 Part 2. Theoretical Analysis 19

3 Part 3. Numerical Results 20

4 Part 4. Discussion 23

5 Part 5. Conclusion 24

1 Part 1. Source Code

In this part, I implemented the numerical experiment in C. The program contains the following main steps:

(1) construct Newton interpolation polynomials for each interval from 2010–2025 to 2024–2025;

(2) extrapolate the corresponding value at 2026;

(3) construct a quadratic least-squares fitting curve as a smoother comparison method;

(4) generate data files and plots by `gnuplot`;

(5) output two types of figures: one for the interpolation/fitting curves and one for the comparison of predictions under different schemes.

The final code used in this report is attached below.

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define TOTAL_YEARS 16
#define MAXN 20
#define CURVE_POINTS 321
#define CASES 15 /* 2010-2025 到 2024-2025 */

typedef struct {
   int year;
   double value;
} DataPoint;

typedef struct {
   int start_year;
   int end_year;
   int n;
   double pred2026;
   double score;
} Result;

/* ---------- 牛顿插值部分 ---------- */
```

```
void build_newton_coeff(double x[], double y[], int n, double coeff
   []) {
   double dd[MAXN][MAXN];
   int i, j;

   for (i = 0; i < n; i++) {
      dd[i][0] = y[i];
   }

   for (j = 1; j < n; j++) {
      for (i = 0; i < n - j; i++) {
         dd[i][j] = (dd[i + 1][j - 1] - dd[i][j - 1]) / (x[i + j] -
            x[i]);
      }
   }

   for (i = 0; i < n; i++) {
      coeff[i] = dd[0][i];
   }
}

double newton_eval(double x[], double coeff[], int n, double t) {
   double result = coeff[0];
   double term = 1.0;
   int i;

   for (i = 1; i < n; i++) {
      term *= (t - x[i - 1]);
      result += coeff[i] * term;
   }

   return result;
}

double calc_score(double pred, double y2023, double y2024, double
   y2025) {
   double recent_trend = y2025 + (y2025 - y2024);
   double smooth_ref = (y2023 + y2024 + y2025) / 3.0;

   double s1 = fabs(pred - recent_trend);
   double s2 = fabs(pred - smooth_ref);
```

```
    if (pred < 0) {
        s1 += 1000000.0;
        s2 += 1000000.0;
    }

    return s1 + 0.5 * s2;
}

void find_best_k(Result results[], int m, int top_idx[], int k) {
    int used[CASES] = {0};
    int i, j, best;

    for (i = 0; i < k; i++) {
        best = -1;
        for (j = 0; j < m; j++) {
            if (!used[j]) {
                if (best == -1 || results[j].score < results[best].score
                    ) {
                    best = j;
                }
            }
        }
        top_idx[i] = best;
        if (best != -1) {
            used[best] = 1;
        }
    }
}

/* ---------- 二次最小二乘拟合部分 ---------- */

void solve_3x3(double A[3][4], double ans[3]) {
    int i, j, k;

    for (i = 0; i < 3; i++) {
        int pivot = i;
        for (j = i + 1; j < 3; j++) {
            if (fabs(A[j][i]) > fabs(A[pivot][i])) {
                pivot = j;
            }
```

```
        }

        if (pivot != i) {
            for (k = 0; k < 4; k++) {
                double tmp = A[i][k];
                A[i][k] = A[pivot][k];
                A[pivot][k] = tmp;
            }
        }

        if (fabs(A[i][i]) < 1e-12) {
            printf("Quadratic fitting failed: singular matrix.\n");
            exit(1);
        }

        for (j = i + 1; j < 3; j++) {
            double factor = A[j][i] / A[i][i];
            for (k = i; k < 4; k++) {
                A[j][k] -= factor * A[i][k];
            }
        }
    }

    for (i = 2; i >= 0; i--) {
        ans[i] = A[i][3];
        for (j = i + 1; j < 3; j++) {
            ans[i] -= A[i][j] * ans[j];
        }
        ans[i] /= A[i][i];
    }
}

void quadratic_fit(double x[], double y[], int n, double coef[3]) {
    double s0 = n;
    double s1 = 0.0, s2 = 0.0, s3 = 0.0, s4 = 0.0;
    double t0 = 0.0, t1 = 0.0, t2 = 0.0;
    int i;

    for (i = 0; i < n; i++) {
        double xi = x[i];
        double yi = y[i];
```

```
        s1 += xi;
        s2 += xi * xi;
        s3 += xi * xi * xi;
        s4 += xi * xi * xi * xi;

        t0 += yi;
        t1 += xi * yi;
        t2 += xi * xi * yi;
    }

    double A[3][4] = {
        {s4, s3, s2, t2},
        {s3, s2, s1, t1},
        {s2, s1, s0, t0}
    };

    solve_3x3(A, coef);
}

double quadratic_eval(double coef[3], double x) {
    return coef[0] * x * x + coef[1] * x + coef[2];
}

/* ---------- 对称对数相关 ---------- */

double signed_log10_transform(double y) {
    if (y > 0.0) return log10(y + 1.0);
    if (y < 0.0) return -log10(fabs(y) + 1.0);
    return 0.0;
}

double max_double(double a, double b) {
    return (a > b) ? a : b;
}

double min_double(double a, double b) {
    return (a < b) ? a : b;
}

/* ---------- 主程序 ---------- */
```

```c
int main() {
    DataPoint data[TOTAL_YEARS] = {
        {2010, 1588},
        {2011, 1600},
        {2012, 1635},
        {2013, 1640},
        {2014, 1687},
        {2015, 1655},
        {2016, 1786},
        {2017, 1723},
        {2018, 1523},
        {2019, 1465},
        {2020, 1200},
        {2021, 1062},
        {2022, 956},
        {2023, 902},
        {2024, 954},
        {2025, 792}
    };

    FILE *fp_actual, *fp_pos_pred, *fp_neg_pred, *fp_best_pred;
    FILE *fp_fit_curve, *fp_fit_pred;
    FILE *fp_scheme_all_pos, *fp_scheme_all_neg;
    FILE *fp_scheme_zoom_pos, *fp_scheme_zoom_neg;
    FILE *fp_gp1, *fp_gp2, *fp_gp3;

    Result results[CASES];

    int result_count = 0;
    int start, i, k;
    double quad_pred2026 = 0.0;

    /* ---------- 真实数据点 ---------- */
    fp_actual = fopen("actual_data.dat", "w");
    if (fp_actual == NULL) {
        printf("Cannot open actual_data.dat\n");
        return 1;
    }
    for (i = 0; i < TOTAL_YEARS; i++) {
        fprintf(fp_actual, "%d %.6f\n", data[i].year, data[i].value);
    }
```

```c
fclose(fp_actual);

/* ---------- 正负预测点分开存 ---------- */
fp_pos_pred = fopen("positive_predictions.dat", "w");
fp_neg_pred = fopen("negative_predictions.dat", "w");
if (fp_pos_pred == NULL || fp_neg_pred == NULL) {
   printf("Cannot open prediction files\n");
   return 1;
}

printf("Interpolation results:\n");
printf("Range N Predicted_2026 Score\n");
printf("
   ------------------------------------------------------------\n")
   ;

for (start = 2010; start <= 2024; start++) {
   int n = 2025 - start + 1;
   double x[MAXN], y[MAXN], coeff[MAXN];
   double pred2026;
   double score;
   char filename[64];
   FILE *fp_curve;

   for (i = 0; i < n; i++) {
      x[i] = (double)i;
      y[i] = data[(start - 2010) + i].value;
   }

   build_newton_coeff(x, y, n, coeff);
   pred2026 = newton_eval(x, coeff, n, (double)n);
   score = calc_score(pred2026, data[13].value, data[14].value,
      data[15].value);

   results[result_count].start_year = start;
   results[result_count].end_year = 2025;
   results[result_count].n = n;
   results[result_count].pred2026 = pred2026;
   results[result_count].score = score;
   result_count++;
```

```c
        if (pred2026 >= 0.0) {
            fprintf(fp_pos_pred, "2026 %.6f \"%d\"\n", pred2026, start)
                ;
        } else {
            fprintf(fp_neg_pred, "2026 %.6f \"%d\"\n", pred2026, start)
                ;
        }

        printf("%d-%d %-6d %-18.6f %.6f\n",
              start, 2025, n, pred2026, score);

        sprintf(filename, "curve_%d_2025.dat", start);
        fp_curve = fopen(filename, "w");
        if (fp_curve == NULL) {
            printf("Cannot open %s\n", filename);
            return 1;
        }

        for (k = 0; k < CURVE_POINTS; k++) {
            double year = (start - 0.5) + (2026.5 - (start - 0.5)) * k
                / (CURVE_POINTS - 1);
            double t = year - start;
            double val = newton_eval(x, coeff, n, t);
            fprintf(fp_curve, "%.6f %.6f\n", year, val);
        }
        fclose(fp_curve);
    }

    fclose(fp_pos_pred);
    fclose(fp_neg_pred);

    /* ---------- 相对不那么极端的几个插值预测 ---------- */
    {
        int top_idx[3];
        fp_best_pred = fopen("best_predictions.dat", "w");
        if (fp_best_pred == NULL) {
            printf("Cannot open best_predictions.dat\n");
            return 1;
        }

        find_best_k(results, result_count, top_idx, 3);
```

```
    printf("\nRelatively less extreme interpolation predictions:\n
        ");
    for (i = 0; i < 3; i++) {
        int idx = top_idx[i];
        if (idx != -1) {
            fprintf(fp_best_pred, "2026 %.6f \"%d\"\n",
                    results[idx].pred2026,
                    results[idx].start_year);

            printf("Top %d: %d-%d -> %.6f\n",
                   i + 1,
                   results[idx].start_year,
                   results[idx].end_year,
                   results[idx].pred2026);
        }
    }
    fclose(fp_best_pred);
}

/* ---------- 二次拟合 ---------- */
{
    double x[TOTAL_YEARS], y[TOTAL_YEARS];
    double coef[3];

    for (i = 0; i < TOTAL_YEARS; i++) {
        x[i] = (double)i;
        y[i] = data[i].value;
    }

    quadratic_fit(x, y, TOTAL_YEARS, coef);
    quad_pred2026 = quadratic_eval(coef, 16.0);

    printf("\nQuadratic least-squares fitting result:\n");
    printf("y = %.6f x^2 + %.6f x + %.6f\n", coef[0], coef[1],
        coef[2]);
    printf("Predicted 2026 by quadratic fit = %.6f\n",
        quad_pred2026);

    fp_fit_curve = fopen("quadratic_fit_curve.dat", "w");
    if (fp_fit_curve == NULL) {
```

```c
        printf("Cannot open quadratic_fit_curve.dat\n");
        return 1;
    }

    for (k = 0; k < CURVE_POINTS; k++) {
        double year = 2009.5 + (2026.5 - 2009.5) * k / (
            CURVE_POINTS - 1);
        double t = year - 2010.0;
        double val = quadratic_eval(coef, t);
        fprintf(fp_fit_curve, "%.6f %.6f\n", year, val);
    }
    fclose(fp_fit_curve);

    fp_fit_pred = fopen("fit_prediction.dat", "w");
    if (fp_fit_pred == NULL) {
        printf("Cannot open fit_prediction.dat\n");
        return 1;
    }
    fprintf(fp_fit_pred, "2026 %.6f \"fit\"\n", quad_pred2026);
    fclose(fp_fit_pred);
}

/* ---------- 生成方案对比数据 ---------- */
double all_log_max = 0.0;
double zoom_raw_min = 1e100, zoom_raw_max = -1e100;

fp_scheme_all_pos = fopen("scheme_compare_all_pos.dat", "w");
fp_scheme_all_neg = fopen("scheme_compare_all_neg.dat", "w");
fp_scheme_zoom_pos = fopen("scheme_compare_zoom_pos.dat", "w");
fp_scheme_zoom_neg = fopen("scheme_compare_zoom_neg.dat", "w");

if (fp_scheme_all_pos == NULL || fp_scheme_all_neg == NULL ||
    fp_scheme_zoom_pos == NULL || fp_scheme_zoom_neg == NULL) {
    printf("Cannot open scheme comparison data files.\n");
    return 1;
}

for (i = 0; i < result_count; i++) {
    int xidx = results[i].start_year - 2010;
    double raw = results[i].pred2026;
    double logv = signed_log10_transform(raw);
```

```
    all_log_max = max_double(all_log_max, fabs(logv));

    if (raw >= 0.0) {
        fprintf(fp_scheme_all_pos, "%d %.6f\n", xidx, logv);
    } else {
        fprintf(fp_scheme_all_neg, "%d %.6f\n", xidx, logv);
    }

    if (results[i].start_year >= 2017 && results[i].start_year <=
        2024) {
        zoom_raw_min = min_double(zoom_raw_min, raw);
        zoom_raw_max = max_double(zoom_raw_max, raw);

        if (raw >= 0.0) {
            fprintf(fp_scheme_zoom_pos, "%d %.6f\n", xidx, raw);
        } else {
            fprintf(fp_scheme_zoom_neg, "%d %.6f\n", xidx, raw);
        }
    }
}

{
    double raw = quad_pred2026;
    double logv = signed_log10_transform(raw);

    all_log_max = max_double(all_log_max, fabs(logv));

    if (raw >= 0.0) {
        fprintf(fp_scheme_all_pos, "15 %.6f\n", logv);
        fprintf(fp_scheme_zoom_pos, "15 %.6f\n", raw);
    } else {
        fprintf(fp_scheme_all_neg, "15 %.6f\n", logv);
        fprintf(fp_scheme_zoom_neg, "15 %.6f\n", raw);
    }

    zoom_raw_min = min_double(zoom_raw_min, raw);
    zoom_raw_max = max_double(zoom_raw_max, raw);
}

fclose(fp_scheme_all_pos);
```

```c
fclose(fp_scheme_all_neg);
fclose(fp_scheme_zoom_pos);
fclose(fp_scheme_zoom_neg);

double main_log_limit = all_log_max + 0.25;
double zoom_raw_margin = 120.0;
double inset_y1 = zoom_raw_min - zoom_raw_margin;
double inset_y2 = zoom_raw_max + zoom_raw_margin;

if (inset_y1 > -1100.0) inset_y1 = -1100.0;
if (inset_y2 < 2000.0) inset_y2 = 2000.0;

/* ---------- 图1: 主图 interpolation_extrapolation.png ----------
   */
fp_gp1 = fopen("plot_main.gnuplot", "w");
if (fp_gp1 == NULL) {
   printf("Cannot open plot_main.gnuplot\n");
   return 1;
}

fprintf(fp_gp1, "set terminal pngcairo size 1600,950 enhanced
   font 'Arial,14'\n");
fprintf(fp_gp1, "set output 'interpolation_extrapolation.png'\n")
   ;
fprintf(fp_gp1, "set title 'Interpolation vs Fitting for 2026
   Prediction'\n");
fprintf(fp_gp1, "set xlabel 'Year'\n");
fprintf(fp_gp1, "set ylabel 'Value'\n");
fprintf(fp_gp1, "set grid\n");
fprintf(fp_gp1, "set key outside\n");
fprintf(fp_gp1, "set xrange [2009.5:2026.8]\n");
fprintf(fp_gp1, "set yrange [0:2000]\n");
fprintf(fp_gp1, "set style data lines\n");

fprintf(fp_gp1, "plot \\\n");
for (start = 2010; start <= 2024; start++) {
   fprintf(fp_gp1,
         "'curve_%d_2025.dat' using 1:2 with lines lw 1 title '%d
            -2025 interp', \\\n",
         start, start);
}
```

```
fprintf(fp_gp1,
      "'quadratic_fit_curve.dat' using 1:2 with lines lw 3 lc rgb
          'black' title 'Quadratic fit curve', \\\n");
fprintf(fp_gp1,
      "'actual_data.dat' using 1:2 with points pt 7 ps 1.8 lc rgb
          'purple' title 'Actual data points', \\\n");
fprintf(fp_gp1,
      "'positive_predictions.dat' using 1:2 with points pt 5 ps
         1.6 lc rgb 'red' title 'Positive interp predictions',
         \\\n");
fprintf(fp_gp1,
      "'positive_predictions.dat' using 1:2:3 with labels offset
         0.5,0.2 tc rgb 'red' notitle, \\\n");
fprintf(fp_gp1,
      "'negative_predictions.dat' using 1:2 with points pt 5 ps
         1.6 lc rgb 'blue' title 'Negative interp predictions',
         \\\n");
fprintf(fp_gp1,
      "'negative_predictions.dat' using 1:2:3 with labels offset
         0.5,-0.4 tc rgb 'blue' notitle, \\\n");
fprintf(fp_gp1,
      "'best_predictions.dat' using 1:2 with points pt 9 ps 2.0
         lc rgb 'orange' title 'Less extreme interp predictions',
          \\\n");
fprintf(fp_gp1,
      "'fit_prediction.dat' using 1:2 with points pt 11 ps 2.5 lc
          rgb 'dark-green' title 'Quadratic-fit prediction', \\\n
         ");
fprintf(fp_gp1,
      "'fit_prediction.dat' using 1:2:3 with labels offset
         0.5,0.8 tc rgb 'dark-green' notitle\n");

fclose(fp_gp1);

/* ---------- 图2: 全部方案主图 (对称对数) ---------- */
fp_gp2 = fopen("plot_bar_main.gnuplot", "w");
if (fp_gp2 == NULL) {
   printf("Cannot open plot_bar_main.gnuplot\n");
   return 1;
}
```

```
fprintf(fp_gp2, "set terminal pngcairo size 1600,950 enhanced
   font 'Arial,14'\n");
fprintf(fp_gp2, "set output 'scheme_comparison_main.png'\n");
fprintf(fp_gp2, "set title 'Comparison of 2026 Predictions by
   Different Schemes'\n");
fprintf(fp_gp2, "set xlabel 'Start year / fitting scheme'\n");
fprintf(fp_gp2, "set ylabel 'Signed log scale of prediction value
   '\n");
fprintf(fp_gp2, "set grid ytics\n");
fprintf(fp_gp2, "set boxwidth 0.72\n");
fprintf(fp_gp2, "set style fill solid 0.85 border -1\n");
fprintf(fp_gp2, "set xtics rotate by -45\n");
fprintf(fp_gp2, "set xrange [-1:16]\n");
fprintf(fp_gp2, "set yrange [%.6f:%.6f]\n", -main_log_limit,
   main_log_limit);

fprintf(fp_gp2,
   "set xtics ('2010' 0, '2011' 1, '2012' 2, '2013' 3, '2014' 4,
      '2015' 5, "
   "'2016' 6, '2017' 7, '2018' 8, '2019' 9, '2020' 10, '2021' 11,
       "
   "'2022' 12, '2023' 13, '2024' 14, 'fit' 15)\n");

fprintf(fp_gp2,
   "set ytics ("
   "'-10^7' -7, '-10^6' -6, '-10^5' -5, '-10^4' -4, '-10^3' -3,
      '-10^2' -2, '-10^1' -1, "
   "'0' 0, "
   "'10^1' 1, '10^2' 2, '10^3' 3, '10^4' 4, '10^5' 5, '10^6' 6,
      '10^7' 7)\n");

fprintf(fp_gp2, "plot \\\n");
fprintf(fp_gp2,
   "'scheme_compare_all_pos.dat' using 1:2 with boxes fc rgb 'red
      ' title 'Positive prediction', \\\n");
fprintf(fp_gp2,
   "'scheme_compare_all_neg.dat' using 1:2 with boxes fc rgb '
      blue' title 'Negative prediction'\n");

fclose(fp_gp2);
```

```c
/* ---------- 图3: 局部放大图 (线性) ---------- */
fp_gp3 = fopen("plot_bar_zoom.gnuplot", "w");
if (fp_gp3 == NULL) {
   printf("Cannot open plot_bar_zoom.gnuplot\n");
   return 1;
}

fprintf(fp_gp3, "set terminal pngcairo size 1200,800 enhanced
   font 'Arial,14'\n");
fprintf(fp_gp3, "set output 'scheme_comparison_zoom.png'\n");
fprintf(fp_gp3, "set title 'Zoom: 2017-2024 and fit'\n");
fprintf(fp_gp3, "set xlabel 'Start year / fitting scheme'\n");
fprintf(fp_gp3, "set ylabel 'Predicted value for 2026'\n");
fprintf(fp_gp3, "set grid ytics\n");
fprintf(fp_gp3, "set boxwidth 0.72\n");
fprintf(fp_gp3, "set style fill solid 0.85 border -1\n");
fprintf(fp_gp3, "set xrange [6.5:15.5]\n");
fprintf(fp_gp3, "set yrange [%.6f:%.6f]\n", inset_y1, inset_y2);
fprintf(fp_gp3,
   "set xtics ('2017' 7, '2018' 8, '2019' 9, '2020' 10, '2021'
      11, '2022' 12, '2023' 13, '2024' 14, 'fit' 15) rotate by
      -45\n");

fprintf(fp_gp3, "plot \\\n");
fprintf(fp_gp3,
   "'scheme_compare_zoom_pos.dat' using 1:2 with boxes fc rgb '
      red' title 'Positive prediction', \\\n");
fprintf(fp_gp3,
   "'scheme_compare_zoom_neg.dat' using 1:2 with boxes fc rgb '
      blue' title 'Negative prediction'\n");

fclose(fp_gp3);

printf("\nFiles generated:\n");
printf("1. actual_data.dat\n");
printf("2. curve_2010_2025.dat ... curve_2024_2025.dat\n");
printf("3. positive_predictions.dat\n");
printf("4. negative_predictions.dat\n");
printf("5. best_predictions.dat\n");
printf("6. quadratic_fit_curve.dat\n");
printf("7. fit_prediction.dat\n");
```

```
    printf("8. scheme_compare_all_pos.dat\n");
    printf("9. scheme_compare_all_neg.dat\n");
    printf("10. scheme_compare_zoom_pos.dat\n");
    printf("11. scheme_compare_zoom_neg.dat\n");
    printf("12. plot_main.gnuplot\n");
    printf("13. plot_bar_main.gnuplot\n");
    printf("14. plot_bar_zoom.gnuplot\n");
    printf("15. interpolation_extrapolation.png\n");
    printf("16. scheme_comparison_main.png\n");
    printf("17. scheme_comparison_zoom.png\n");

    system("gnuplot plot_main.gnuplot");
    system("gnuplot plot_bar_main.gnuplot");
    system("gnuplot plot_bar_zoom.gnuplot");

    printf("\nDone.\n");
    return 0;
}
```

2 Part 2. Theoretical Analysis

Suppose that for a given interval we have points

$$x_0, x_1, \ldots, x_{n-1}$$

and the corresponding values

$$y_0, y_1, \ldots, y_{n-1}.$$

In this report, I use Newton interpolation to construct the polynomial

$$P_{n-1}(x),$$

which satisfies

$$P_{n-1}(x_i) = y_i, \qquad i = 0, 1, \ldots, n-1.$$

The Newton form can be written as

$$P_{n-1}(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots + a_{n-1} \prod_{k=0}^{n-2} (x - x_k),$$

where the coefficients $a_0, a_1, \ldots, a_{n-1}$ are obtained from divided differences.

For numerical convenience, I do not directly use the absolute years in each interpolation window. Instead, I use relative coordinates. For example, in the interval 2010–2025, I let

$$2010 \to 0,\ 2011 \to 1,\ \ldots,\ 2025 \to 15,$$

so the predicted year 2026 corresponds to

$$x = 16.$$

Similarly, in the interval 2020–2025, I let

$$2020 \to 0,\ 2021 \to 1,\ \ldots,\ 2025 \to 5,$$

and then evaluate the polynomial at

$$x = 6.$$

This treatment reduces numerical instability compared with using absolute year values directly.

However, interpolation and extrapolation are very different. Interpolation ensures that the curve passes through all known points exactly, but extrapolation outside the known interval is often unstable, especially when:

(1) the polynomial degree is high;

(2) the data contain local fluctuations and turning points;

(3) the predicted point lies outside the interpolation interval.

This is exactly the main phenomenon that the teacher intended us to observe.

Besides interpolation, I also use a quadratic least-squares fitting curve

$$f(x) = ax^2 + bx + c,$$

which does not pass through every point exactly, but usually reflects the overall trend more smoothly. Therefore, it serves as a comparison method for the interpolation-based predictions.

3 Part 3. Numerical Results

After running the program, I obtained the following predicted values for 2026:

Range	Number of points	Method	Predicted value for 2026
2010–2025	16	Interpolation	1164942
2011–2025	15	Interpolation	728158
2012–2025	14	Interpolation	404369
2013–2025	13	Interpolation	201697
2014–2025	12	Interpolation	89816
2015–2025	11	Interpolation	34765
2016–2025	10	Interpolation	10803
2017–2025	9	Interpolation	1864
2018–2025	8	Interpolation	-715
2019–2025	7	Interpolation	-1013
2020–2025	6	Interpolation	-686
2021–2025	5	Interpolation	-278
2022–2025	4	Interpolation	96
2023–2025	3	Interpolation	416
2024–2025	2	Interpolation	630
2010–2025	16	Quadratic fitting	495.79

From the table, several features can be seen immediately:

(1) When the interpolation interval starts very early, the predicted values are extremely large. For example, the result from 2010–2025 is 1164942, which is completely outside the scale of the original data.

(2) Some intervals give negative predictions, such as -715, -1013, -686, and -278. For birth-population data, such results are obviously unreasonable in practice.

(3) When only the latest few years are used, such as 2022–2025, 2023–2025, and 2024–2025, the predicted values become moderate again.

(4) The quadratic fitting result, about 495.79, lies in the same rough range as the milder recent-window interpolation results.

The generated figures also show this behavior very clearly.

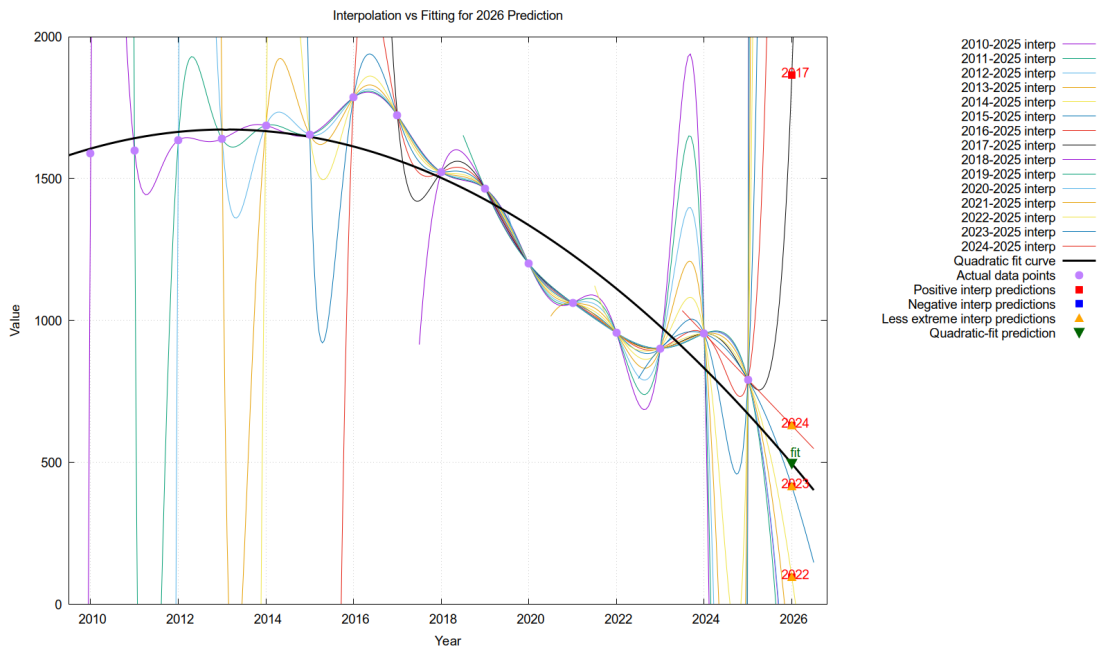

图 2: Interpolation curves, actual data points, and predicted points for 2026.

Figure 2 compares all interpolation curves with the quadratic fitting curve. It shows that high-order interpolation curves can oscillate strongly near the right end of the interval, and the predicted points at 2026 vary dramatically from one scheme to another. In contrast, the quadratic fitting curve is much smoother and produces a more moderate prediction.

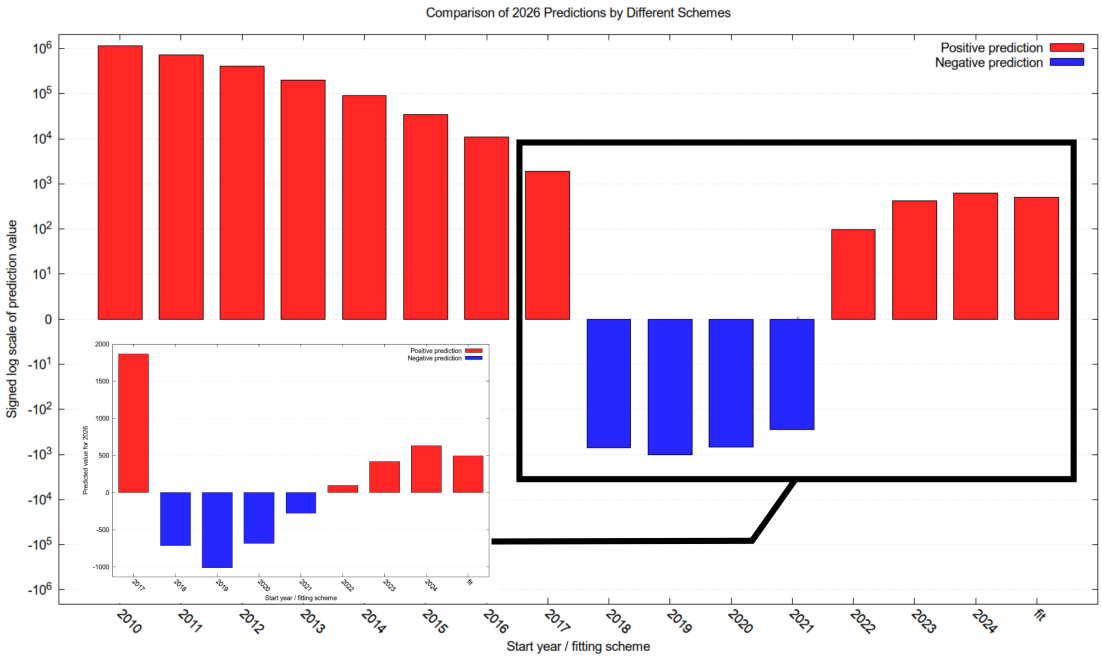

图 3: Comparison of all prediction schemes using a signed logarithmic scale.

Figure 3 uses a signed logarithmic scale to display all schemes together. In this figure, both very large positive values and negative values can be shown in one coordinate system.

This makes it much easier to see how strongly the early-window interpolation results deviate from the rest.

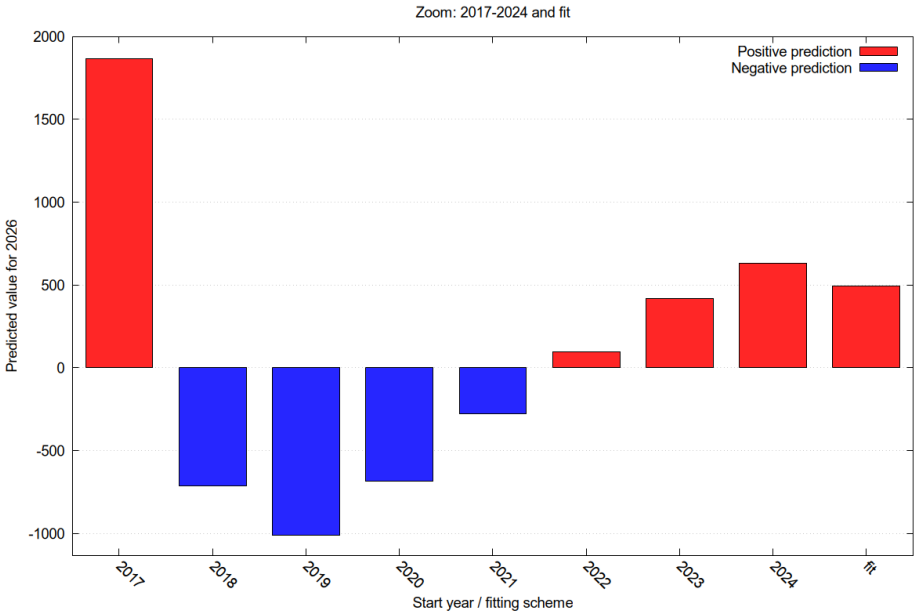

图 4: Zoomed comparison of the predictions from 2017–2024 and quadratic fitting.

Figure 4 focuses on the schemes from 2017–2024 and the quadratic fitting result. After removing the visually overwhelming effect of the extremely large values, the differences between the more moderate prediction schemes can be observed more clearly.

4 Part 4. Discussion

From this experiment, I think the most important conclusion is that interpolation and extrapolation are fundamentally different problems.

Inside the interpolation interval, Newton interpolation works well in the sense that it passes through all known points exactly. However, once I use the polynomial to predict the value at 2026, the result becomes highly sensitive to the choice of interpolation window. When the interval is long, the polynomial degree is high, so the extrapolation becomes extremely unstable. This is why the schemes from 2010–2025 to 2016–2025 give unrealistically huge positive values.

Another striking phenomenon is that some interpolation schemes give negative predictions. Mathematically, this is not an error in the code; instead, it is a typical consequence of polynomial oscillation outside the interpolation interval. But physically, birth population cannot be negative, so these results have no practical meaning. This shows that a mathematically valid extrapolation formula does not necessarily give a practically meaningful forecast.

I also noticed that the original data are not monotonic. From 2010 to around 2016, the values remain relatively high, while after that they decrease significantly. There is even a small rebound in 2024 before the value drops again in 2025. Because of this turning behavior, a high-degree polynomial tries to pass through every local fluctuation, which makes the extrapolation near the endpoint even more unstable. In my opinion, this is the direct reason why the predicted values differ so much among different windows.

Compared with interpolation, the quadratic least-squares fitting method does not force the curve to pass through every point exactly. As a result, it loses some local detail, but gains stability. The fitted result 495.79 is not necessarily the true future value, but it is much easier to accept as a trend-based reference than values such as 1164942 or -1013. Therefore, for real data with visible fluctuations, fitting is usually more suitable than direct high-order interpolation if the goal is forecasting rather than exact reproduction of the known data.

From a student's point of view, I think this experiment is very instructive. At first, it may seem natural that using more data points should give a better prediction. But this experiment shows that this intuition is not always correct. In fact, using too many points in a single high-order polynomial may make the future prediction much worse. So the key lesson is not simply "use more data," but rather "choose a reasonable model."

5 Part 5. Conclusion

In this exercise, I extended the original classroom example of polynomial extrapolation to the birth-population data from 2010 to 2025, and predicted the value in 2026 using multiple interpolation schemes. For each interval from 2010–2025 to 2024–2025, I constructed a Newton interpolation polynomial and compared the resulting extrapolated values. I also introduced a quadratic least-squares fitting curve as a smoother reference.

The numerical results show that high-order interpolation extrapolation is extremely unstable. Some schemes give unrealistically huge positive values, while some others even produce negative values. This clearly demonstrates the hazard of extrapolation. In contrast, the quadratic fitting result is much more moderate and visually more consistent with the recent trend of the data.

Therefore, my main conclusions are:

(1) high-order interpolation can reproduce known data exactly, but its extrapolation may be very unreliable;

(2) the prediction is highly sensitive to the choice of interpolation window;

(3) for fluctuating real-world data, fitting methods are usually more stable than direct high-order interpolation when predicting future values;

(4) this experiment successfully illustrates the main idea emphasized in class: extrapolation should be treated with great caution.